

Paradigmas de Programación

Razonamiento ecuacional
Inducción estructural

2do cuatrimestre de 2026
Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Introducción

Inducción estructural

Extensionalidad

Isomorfismos de tipos

Casos de estudio

“En verdad el mago le había dicho eso una vez, pero Ged no le había hecho mucho caso; aunque ahora sabía que Ogión nunca le diría nada sin alguna buena razón.”

Un mago de Terramar
Úrsula K. Le Guin



De dónde venimos

Dos deudas de las clases anteriores

- ▶ En la clase 1 dijimos que el $=$ es **equivalencia denotacional**.
 - ▶ Vale en las dos direcciones.
 - ▶ No depende de la ejecución.
- ▶ En la clase 2 dijimos que la recursión estructural **admite un principio de prueba**.
 - ▶ Ese era el pago de la disciplina.
- ▶ Hoy cobramos las dos.

La pregunta con la que cerramos la clase 1

- ▶ Si $f\ x + f\ x = 2 * f\ x$ vale siempre en Haskell, ¿qué otras cosas de las que sabemos de matemática podemos usar?
- ▶ Respuesta: **casi todas**.
- ▶ Podemos razonar sobre programas como razonamos sobre ecuaciones.
 - ▶ Las ecuaciones **son** el programa.
- ▶ A eso se lo llama **razonamiento ecuacional**.

Las ecuaciones se leen para los dos lados

`doble x = x + x`

- ▶ Para **ejecutar** la usamos de izquierda a derecha: es reducción.
- ▶ Para **razonar** la usamos en cualquier dirección: es equivalencia.
- ▶ Esa libertad es exactamente lo que un lenguaje con efectos no permite.
- ▶ Es la contracara de lo que vimos con `f(3) + f(3)` en Python.

Motivación

Queremos demostrar que ciertas expresiones son equivalentes.
¿Para qué?

Para justificar que un algoritmo es correcto

Por ejemplo, si logramos demostrar que:

$$\forall xs :: [Int]. \text{quickSort } xs = \text{insertionSort } xs$$

esto nos da confianza relativa de un algoritmo con respecto al otro.

Para posibilitar optimizaciones

Por ejemplo, reemplazar $f(3) + f(3)$ por $2 * f(3)$.

Como ya vimos, en un lenguaje funcional podemos hacerlo.

En un lenguaje imperativo **no**, ya que f puede tener efectos.

Hipótesis de trabajo

Para razonar sobre equivalencia de expresiones vamos a suponer:

1. Que trabajamos con estructuras de datos **finitas**.

Técnicamente: con tipos de datos inductivos.

2. Que trabajamos con **funciones totales**¹.

- ▶ Las ecuaciones deben cubrir todos los casos.
- ▶ La recursión siempre debe terminar.

3. Que el programa **no depende del orden** de las ecuaciones.

<code>vacía [] = True</code>	$\rightsquigarrow$	<code>vacía [] = True</code>
<code>vacía _ = False</code>		<code>vacía (_ : _) = False</code>

Relajar estas hipótesis es posible pero más complejo. No es tema de hoy.

¹En la práctica se verán técnicas para demostrar propiedades que involucren funciones parciales, siempre para los casos en los que devuelven un valor.

Igualdades por definición

Principio de reemplazo

Sea $e1 = e2$ una ecuación incluida en el programa.

Las siguientes operaciones preservan la igualdad de expresiones:

1. Reemplazar **cualquier instancia** de $e1$ por $e2$.
2. Reemplazar **cualquier instancia** de $e2$ por $e1$.

Si una igualdad se puede demostrar usando solamente el principio de reemplazo, decimos que la igualdad vale **por definición**.

Ejemplo: principio de reemplazo

Le damos nombre a las ecuaciones del programa:

```
sucesor :: Int -> Int  
{SUC} sucesor n = n + 1
```

```
      sucesor (factorial 10) + 1  
=    (factorial 10 + 1) + 1      {SUC}  
=    sucesor (factorial 10 + 1) {SUC}
```

Igualdades por definición

Ejemplo: principio de reemplazo

{L0} `length [] = 0`

{L1} `length (_ : xs) = 1 + length xs`

{S0} `suma [] = 0`

{S1} `suma (x : xs) = x + suma xs`

Veamos que `length ["a", "b"] = suma [1, 1]`:

	<code>length ["a", "b"]</code>	
<code>=</code>	<code>1 + length ["b"]</code>	{L1}
<code>=</code>	<code>1 + (1 + length [])</code>	{L1}
<code>=</code>	<code>1 + (1 + 0)</code>	{L0}
<code>=</code>	<code>1 + (1 + suma [])</code>	{S0}
<code>=</code>	<code>1 + suma [1]</code>	{S1}
<code>=</code>	<code>suma [1, 1]</code>	{S1}

Introducción

Inducción estructural

Extensionalidad

Isomorfismos de tipos

Casos de estudio

Inducción sobre booleanos

El principio de reemplazo no alcanza para probar todas las equivalencias que nos interesan.

Ejemplo

`{NT} not True = False`

`{NF} not False = True`

¿Podemos probar $\forall x :: \text{Bool}. \text{not } (\text{not } x) = x$?

El problema es que la expresión

$$\text{not } (\text{not } x)$$

está “trabada”: no se puede aplicar ninguna ecuación.

Inducción sobre booleanos

Principio de inducción sobre booleanos

Si $\mathcal{P}(\text{True})$ y $\mathcal{P}(\text{False})$ entonces $\forall x :: \text{Bool}. \mathcal{P}(x)$.

Ejemplo

{NT} $\text{not True} = \text{False}$

{NF} $\text{not False} = \text{True}$

Para probar $\forall x :: \text{Bool}. \text{not} (\text{not } x) = x$
basta probar:

1. $\text{not} (\text{not True}) = \text{True}$.

$$\begin{array}{ccccccc} \text{not} & (\text{not True}) & = & \text{not False} & = & \text{True} \\ & \uparrow & & \uparrow & & \\ & \text{NT} & & \text{NF} & & \end{array}$$

2. $\text{not} (\text{not False}) = \text{False}$.

$$\begin{array}{ccccccc} \text{not} & (\text{not False}) & = & \text{not True} & = & \text{False} \\ & \uparrow & & \uparrow & & \\ & \text{NF} & & \text{NT} & & \end{array}$$

Inducción sobre naturales

```
data Nat = Zero | Suc Nat
```

Principio de inducción sobre naturales

Si $\mathcal{P}(\text{Zero})$ y $\forall n :: \text{Nat}. \left(\underbrace{\mathcal{P}(n)}_{\text{hipótesis inductiva}} \Rightarrow \underbrace{\mathcal{P}(\text{Suc } n)}_{\text{tesis inductiva}} \right)$,
entonces $\forall n :: \text{Nat}. \mathcal{P}(n)$.

Inducción sobre naturales

Ejemplo

{S0} suma Zero m = m

{S1} suma (Suc n) m = Suc (suma n m)

Para probar $\forall n :: \text{Nat. suma } n \text{ Zero} = n$
basta probar:

1. suma Zero Zero = Zero.

Inmediato por **S0**.

2.

$\forall n :: \text{Nat. } \underbrace{\text{suma } n \text{ Zero} = n}_{\text{H.I.}} \Rightarrow \underbrace{\text{suma (Suc } n) \text{ Zero} = \text{Suc } n}_{\text{T.I.}}$

suma (Suc n) Zero = Suc (suma n Zero) = Suc n
 $\uparrow$ $\uparrow$
 S1 H.I.

Inducción sobre pares

Principio de inducción sobre pares

Si $\forall x :: a. \forall y :: b. \mathcal{P}((x, y))$
entonces $\forall p :: (a, b). \mathcal{P}(p)$.

Ejemplo

{FST} $\text{fst } (x, _) = x$

{SND} $\text{snd } (_, y) = y$

{SWAP} $\text{swap } (x, y) = (y, x)$

Para probar $\forall p :: (a, b). \text{fst } p = \text{snd } (\text{swap } p)$ basta probar:

► $\forall x :: a. \forall y :: b. \text{fst } (x, y) = \text{snd } (\text{swap } (x, y))$

$$\text{fst } (x, y) \overset{\text{FST}}{=} x \overset{\text{SND}}{=} \text{snd } (y, x) \overset{\text{SWAP}}{=} \text{snd } (\text{swap } (x, y))$$

¿Qué habría que hacer para probar una propiedad sobre las listas?

¿Se podrá generalizar?

Inducción estructural

En el **caso general**, tenemos un tipo de datos inductivo:

```
data T = CBase1 ⟨parámetros⟩  
      ...  
      | CBasen ⟨parámetros⟩  
      | CRecursoivo1 ⟨parámetros⟩  
      ...  
      | CRecursoivom ⟨parámetros⟩
```

Principio de inducción estructural

Sea $\mathcal{P}$ una propiedad acerca de las expresiones tipo T tal que:

- ▶ $\mathcal{P}$ vale sobre todos los constructores base de T ,
- ▶ $\mathcal{P}$ vale sobre todos los constructores recursivos de T ,
suponiendo como hipótesis inductiva que vale para los
parámetros de tipo T (es decir, **si** vale $\mathcal{P}(Arg_1), \dots, \mathcal{P}(Arg_k)$
entonces vale $\mathcal{P}(C\ Arg_1 \ \dots\ Arg_k)$ para cada constructor C y sus
argumentos recursivos $Arg_1, \dots, Arg_k$),

entonces $\forall x :: T. \ \mathcal{P}(x)$.

Inducción estructural

Ejemplo: principio de inducción sobre listas

data [a] = [] | a : [a]

Sea $\mathcal{P}$ una propiedad sobre expresiones de tipo [a] tal que:

► $\mathcal{P}([])$

► $\forall x :: a. \forall xs :: [a]. \underbrace{(\mathcal{P}(xs))}_{\text{H.I.}} \Rightarrow \underbrace{\mathcal{P}(x : xs)}_{\text{T.I.}}$

Entonces $\forall xs :: [a]. \mathcal{P}(xs)$.

Ejemplo: principio de inducción sobre árboles binarios

data AB a = Nil | Bin (AB a) a (AB a)

Sea $\mathcal{P}$ una propiedad sobre expresiones de tipo AB a tal que:

► $\mathcal{P}(\text{Nil})$

► $\forall i :: AB a. \forall r :: a. \forall d :: AB a.$

$\underbrace{((\mathcal{P}(i) \wedge \mathcal{P}(d)))}_{\text{H.I.}} \Rightarrow \underbrace{\mathcal{P}(\text{Bin } i \text{ } r \text{ } d))}_{\text{T.I.}}$

Entonces $\forall x :: AB a. \mathcal{P}(x)$.

Inducción estructural

Ejemplo: principio de inducción sobre polinomios

```
data Poli a = X
            | Cte a
            | Suma (Poli a) (Poli a)
            | Prod (Poli a) (Poli a)
```

Sea $\mathcal{P}$ una propiedad sobre expresiones de tipo `Poli a` tal que:

► $\mathcal{P}(X)$

► $\forall k :: a. \mathcal{P}(\text{Cte } k)$

► $\forall p :: \text{Poli } a. \forall q :: \text{Poli } a.$

$$\underbrace{((\mathcal{P}(p) \wedge \mathcal{P}(q))}_{\text{H.I.}} \Rightarrow \underbrace{\mathcal{P}(\text{Suma } p \text{ } q))}_{\text{T.I.}}$$

► $\forall p :: \text{Poli } a. \forall q :: \text{Poli } a.$

$$\underbrace{((\mathcal{P}(p) \wedge \mathcal{P}(q))}_{\text{H.I.}} \Rightarrow \underbrace{\mathcal{P}(\text{Prod } p \text{ } q))}_{\text{T.I.}}$$

Entonces $\forall x :: \text{Poli } a. \mathcal{P}(x).$

Pasos a seguir

- ▶ Leer la propiedad, entenderla y convencerse de que es verdadera.
- ▶ Plantear la propiedad como predicado unario.
- ▶ Plantear el esquema de inducción.
- ▶ Plantear y resolver el o los caso(s) base (justificando cada paso).
- ▶ Plantear y resolver el o los caso(s) inductivo(s) (justificando cada paso y marcando dónde se usó la **Hipótesis Inductiva**).

Esto último es importante. Si no se usó al menos una HI, no hacía falta usar inducción.

Tener en cuenta que cada caso inductivo tiene su propia HI.

Ejemplo: inducción sobre listas

{M0} `map f [] = []`

{M1} `map f (x : xs) = f x : map f xs`

{A0} `[] ++ ys = ys`

{A1} `(x : xs) ++ ys = x : (xs ++ ys)`

Propiedad. $\forall f :: a \rightarrow b. \forall xs :: [a]. \forall ys :: [a].$

`map f (xs ++ ys) = map f xs ++ map f ys`

$\mathcal{P}(xs) \equiv \forall f :: a \rightarrow b. \forall ys : [a].$

`map f (xs ++ ys) = map f xs ++ map f ys`

Por inducción en la estructura de `xs`, basta ver:

1. Caso base, $\mathcal{P}([])$.
2. Caso inductivo, $\forall x :: a. \forall xs :: [a]. (\mathcal{P}(xs) \Rightarrow \mathcal{P}(x : xs))$.

Ejemplo: inducción sobre listas

Caso base:

$$\begin{aligned} & \text{map } f \text{ } ([] ++ \text{ys}) \\ = & \text{map } f \text{ } \text{ys} && \{\text{A0}\} \\ = & [] ++ \text{map } f \text{ } \text{ys} && \{\text{A0}\} \\ = & \text{map } f \text{ } [] ++ \text{map } f \text{ } \text{ys} && \{\text{M0}\} \end{aligned}$$

Caso inductivo:

H.I. $\equiv \forall f :: a \rightarrow b. \forall \text{ys} :: [a].$

$$\text{map } f \text{ } (\text{xs} ++ \text{ys}) = \text{map } f \text{ } \text{xs} ++ \text{map } f \text{ } \text{ys}$$

$$\begin{aligned} & \text{map } f \text{ } ((x : \text{xs}) ++ \text{ys}) \\ = & \text{map } f \text{ } (x : (\text{xs} ++ \text{ys})) && \{\text{A1}\} \\ = & f \text{ } x : \text{map } f \text{ } (\text{xs} ++ \text{ys}) && \{\text{M1}\} \\ = & f \text{ } x : (\text{map } f \text{ } \text{xs} ++ \text{map } f \text{ } \text{ys}) && \{\text{H.I.}\} \\ = & (f \text{ } x : \text{map } f \text{ } \text{xs}) ++ \text{map } f \text{ } \text{ys} && \{\text{A1}\} \\ = & \text{map } f \text{ } (x : \text{xs}) ++ \text{map } f \text{ } \text{ys} && \{\text{M1}\} \end{aligned}$$

Ejemplo: relación entre foldr y foldl

Propiedad. Si $f :: a \rightarrow b \rightarrow b$, $z :: b$, $xs :: [a]$, entonces:

$$\text{foldr } f \ z \ xs = \text{foldl } (\text{flip } f) \ z \ (\text{reverse } xs)$$

$$\mathcal{P}(xs) = \forall f :: a \rightarrow b \rightarrow b, z :: b. \text{foldr } f \ z \ xs = \text{foldl } \dots$$

Por inducción en la estructura de xs . El caso base $\mathcal{P}([])$ es fácil.

Caso inductivo, $\forall x :: a. \forall xs :: [a]. (\mathcal{P}(xs) \Rightarrow \mathcal{P}(x : xs))$:

$$\begin{aligned} & \text{foldr } f \ z \ (x : xs) \\ = & f \ x \ (\text{foldr } f \ z \ xs) && (\text{Def. foldr}) \\ = & f \ x \ (\text{foldl } (\text{flip } f) \ z \ (\text{reverse } xs)) && (\text{H.I.}) \\ = & \text{flip } f \ (\text{foldl } (\text{flip } f) \ z \ (\text{reverse } xs)) \ x && (\text{Def. flip}) \\ = & \text{foldl } (\text{flip } f) \ z \ (\text{reverse } xs ++ [x]) && \text{???} \\ = & \text{foldl } (\text{flip } f) \ z \ (\text{reverse } (x : xs)) && (\text{Def. reverse}) \end{aligned}$$

Para justificar el paso faltante **???**, se puede demostrar:

Lema. Si $g :: b \rightarrow a \rightarrow b$, $z :: b$, $x :: a$, $xs :: [a]$, entonces:

$$\text{foldl } g \ z \ (xs ++ [x]) = g \ (\text{foldl } g \ z \ xs) \ x$$

Lemas de generación

A veces no necesitamos la Hipótesis Inductiva, sino solamente saber que un valor **tiene** alguna de las formas posibles.

Toda $xs :: [a]$ finita cumple:

$xs = []$ o bien $xs = y : ys$ para algún y, ys .

- ▶ Se llaman **lemas de generación**.
 - ▶ Salen de la declaración del tipo.
- ▶ Sirven para hacer análisis por casos sin inducción.

Lemas de generación

Usando el principio de inducción estructural, se puede probar:

Lema de generación para pares

Si $p :: (a, b)$, entonces $\exists x :: a. \exists y :: b. p = (x, y)$.

```
data Either a b = Left a | Right b
```

Lema de generación para sumas

Si $e :: \text{Either } a \ b$, entonces:

- ▶ o bien $\exists x :: a. e = \text{Left } x$
- ▶ o bien $\exists y :: b. e = \text{Right } y$

Introducción

Inducción estructural

Extensionalidad

Isomorfismos de tipos

Casos de estudio

Puntos de vista intensional vs. extensional

¿Vale la siguiente equivalencia de expresiones?

```
quickSort = insertionSort
```

Depende del punto de vista:

Punto de vista intensional. (va con “s”)

Dos valores son iguales si están contruidos de la misma manera.

Punto de vista extensional.

Dos valores son iguales si son indistinguibles al observarlos.

Ejemplo

quickSort e insertionSort

- ▶ **no** son **intensionalmente** iguales;
- ▶ **sí** son **extensionalmente** iguales: computan la misma función.

Principio de extensionalidad funcional

Sean $f, g :: a \rightarrow b$.

Propiedad inmediata

Si $f = g$ entonces $(\forall x :: a. f\ x = g\ x)$.

Principio de extensionalidad funcional

Si $(\forall x :: a. f\ x = g\ x)$ entonces $f = g$.

Principio de extensionalidad funcional

Ejemplo: extensionalidad funcional

{I} $\text{id } x = x$ **{C}** $(g \ . \ f) \ x = g \ (f \ x)$

{S} $\text{swap } (x, y) = (y, x)$

Veamos que $\text{swap} \ . \ \text{swap} = \text{id} :: (a, b) \rightarrow (a, b)$.

Por extensionalidad funcional, basta ver:

$$\forall p :: (a, b). (\text{swap} \ . \ \text{swap}) \ p = \text{id } p$$

Por inducción sobre pares, basta ver:

$$\forall x :: a. \ \forall y :: b. (\text{swap} \ . \ \text{swap}) \ (x, y) = \text{id } (x, y)$$

En efecto: $(\text{swap} \ . \ \text{swap}) \ (x, y)$

$$= \text{swap } (\text{swap } (x, y)) \quad \mathbf{\{C\}}$$

$$= \text{swap } (y, x) \quad \mathbf{\{S\}}$$

$$= (x, y) \quad \mathbf{\{S\}}$$

$$= \text{id } (x, y) \quad \mathbf{\{I\}}$$

Resumen: razonamiento ecuacional

Razonamos ecuacionalmente usando tres principios:

1. Principio de reemplazo

Si el programa declara que $e1 = e2$, cualquier instancia de $e1$ es igual a la correspondiente instancia de $e2$, y viceversa (es decir, $e1 = e2$).

2. Principio de inducción estructural

Para probar $\mathcal{P}$ sobre todas las instancias de un tipo T , basta probar $\mathcal{P}$ para cada uno de los constructores (asumiendo la H.I. para los constructores recursivos).

3. Principio de extensionalidad funcional

Para probar que dos funciones son iguales, basta probar que son iguales punto a punto.

Corrección del razonamiento ecuacional

Supongamos que logramos demostrar que $e1 = e2$.
¿Qué nos asegura eso sobre $e1$ y $e2$?

Cuidado: no necesariamente dan el mismo resultado

Por ejemplo, se puede demostrar:

$$\text{quickSort} = \text{insertionSort}$$

pero `quickSort` e `insertionSort` no dan el mismo resultado.

Corrección con respecto a observaciones

Si demostramos $e1 = e2 :: A$, entonces:

$$\text{obs } e1 \rightsquigarrow \text{True} \quad \text{si y solo si} \quad \text{obs } e2 \rightsquigarrow \text{True}$$

para toda posible “observación” $\text{obs} :: A \rightarrow \text{Bool}$.

Demostración de desigualdades

¿Cómo demostramos que **no** vale una igualdad $e1 = e2 :: A$?

Por la contrarrecíproca de la anterior, basta con encontrar una observación $obs :: A \rightarrow Bool$ que las distinga.

Ejemplo

Demostrar que **no** vale la igualdad:

$id = swap :: (Int, Int) \rightarrow (Int, Int)$

$obs :: ((Int, Int) \rightarrow (Int, Int)) \rightarrow Bool$
 $obs\ f = fst\ (f\ (1, 2)) == 1$

$obs\ id \rightsquigarrow True$
 $obs\ swap \rightsquigarrow False$

Introducción

Inducción estructural

Extensionalidad

Isomorfismos de tipos

Casos de estudio

Misma información, distinta forma

¿Qué relación hay entre los siguientes valores?

```
("hola", (1, True)) :: (String, (Int, Bool))  
((True, "hola"), 1) :: ((Bool, String), Int)
```

Representan la misma información, pero escrita de distinta manera.

Podemos transformar los valores de un tipo en valores del otro:

```
f :: (String, (Int, Bool)) -> ((Bool, String), Int)  
f (s, (i, b)) = ((b, s), i)
```

```
g :: ((Bool, String), Int) -> (String, (Int, Bool))  
g ((b, s), i) = (s, (i, b))
```

Se puede demostrar que:

$$g \circ f = \text{id} \qquad f \circ g = \text{id}$$

Isomorfismos de tipos

Definición

Decimos que dos tipos de datos A y B son **isomorfos** si:

1. Hay una función $f :: A \rightarrow B$ total.
2. Hay una función $g :: B \rightarrow A$ total.
3. Se puede demostrar que $g \circ f = \text{id} :: A \rightarrow A$.
4. Se puede demostrar que $f \circ g = \text{id} :: B \rightarrow B$.

Escribimos $A \simeq B$ para indicar que A y B son isomorfos.

Ejemplo de isomorfismo: currificación

Ejemplo

Veamos que $((a, b) \rightarrow c) \simeq (a \rightarrow b \rightarrow c)$.

```
curry :: ((a, b) -> c) -> a -> b -> c
curry f x y = f (x, y)
```

```
uncurry :: (a -> b -> c) -> (a, b) -> c
uncurry f (x, y) = f x y
```

Ejemplo de isomorfismo: currificación

Veamos que

$\text{uncurry} \cdot \text{curry} = \text{id} \quad :: (a, b) \rightarrow c \rightarrow (a, b) \rightarrow c$

Por extensionalidad funcional, basta ver que si $f :: (a, b) \rightarrow c$:

$(\text{uncurry} \cdot \text{curry}) f = \text{id} f \quad :: (a, b) \rightarrow c$

Por extensionalidad funcional, basta ver que si $p :: (a, b)$:

$(\text{uncurry} \cdot \text{curry}) f p = \text{id} f p \quad :: c$

Por inducción sobre pares, basta ver que si $x :: a, y :: b$:

$(\text{uncurry} \cdot \text{curry}) f (x, y) = \text{id} f (x, y) \quad :: c$

En efecto:

$$\begin{aligned} & (\text{uncurry} \cdot \text{curry}) f (x, y) \\ = & \text{uncurry} (\text{curry} f) (x, y) && (\text{Def. } (..)) \\ = & \text{curry} f x y && (\text{Def. uncurry}) \\ = & f (x, y) && (\text{Def. curry}) \\ = & \text{id} f (x, y) && (\text{Def. id}) \end{aligned}$$

(Y vale también $\text{curry} \cdot \text{uncurry} = \text{id}$).

Más isomorfismos de tipos

$$(a, b) \simeq (b, a)$$

$$(a, (b, c)) \simeq ((a, b), c)$$

$$a \rightarrow b \rightarrow c \simeq b \rightarrow a \rightarrow c$$

$$a \rightarrow (b, c) \simeq (a \rightarrow b, a \rightarrow c)$$

$$\text{Either } a \rightarrow b \rightarrow c \simeq (a \rightarrow c, b \rightarrow c)$$

Introducción

Inducción estructural

Extensionalidad

Isomorfismos de tipos

Casos de estudio

Ejemplo — Necesidad de usar lemas auxiliares

Considerando las definiciones usuales para `(.)` y `(++)` y la siguiente para `reverse`:

{R0} `reverse [] = []`

{R1} `reverse (x : xs) = reverse xs ++ [x]`

Consideremos además la siguiente definición:

`ceros :: [a] -> [Int]`

{Z0} `ceros [] = []`

{Z1} `ceros (_ : xs) = 0 : ceros xs`

Demostremos que `ceros . reverse = reverse . ceros`.

¿Qué ocurre?

Necesitamos un **lema auxiliar**:

$\forall xs\ ys :: [a].\ ceros\ (xs\ ++\ ys) = ceros\ xs\ ++\ ceros\ ys$

Reconocer qué lema falta es la parte difícil.

Se aprende con la práctica.

Ejemplo — Necesidad de generalizar el predicado inductivo

Consideremos la siguiente definición, usando recursión a la cola:

```
suma :: Int -> [Int] -> Int
{S0} suma k []          = k
{S1} suma k (x : xs) = suma (x + k) xs
```

Sean $n :: \text{Int}$ e $ys :: [\text{Int}]$, demostremos que vale:

$$\forall xs :: [\text{Int}]. \text{suma } n \ (xs ++ ys) = \text{suma } (\text{suma } n \ xs) \ ys$$

¿Qué ocurre?

Necesitamos **generalizar** el predicado inductivo de $\mathcal{P}$ a $\mathcal{Q}$:

$$\mathcal{P}(xs) \equiv \boxed{\text{suma } n \ (xs ++ ys) = \text{suma } (\text{suma } n \ xs) \ ys}$$
$$\mathcal{Q}(xs) \equiv \boxed{\forall k :: \text{Int}. \text{suma } k \ (xs ++ ys) = \text{suma } (\text{suma } k \ xs) \ ys}$$

Ejemplo — Necesidad de generalizar el predicado inductivo

Esta definición de la función para pasar de una lista de dígitos decimales al número correspondiente es muy ineficiente:

{N0} número [] = 0

{N1} número (d : ds) = d * 10^{length ds} + número ds

Pensar por qué, y qué tipo de recursión utiliza. Nos ofrecen una versión más eficiente definida con recursión a la cola:

{A0} númeroAc ac [] = ac

{A1} númeroAc ac (d:ds) = númeroAc (10 * ac + d) ds

Y nos aseguran que `númeroAc 0 = número`. ¿La aceptamos?

Demostremos que es cierto.

¿Qué ocurre?

Necesitamos **generalizar** el predicado inductivo de $\mathcal{P}$ a $\mathcal{Q}$:

$$\mathcal{P}(xs) \equiv \boxed{\text{númeroAc } 0 \text{ } xs = \text{número } xs}$$

$$\mathcal{Q}(xs) \equiv \boxed{\forall ac :: \text{Int. } \text{númeroAc } ac \text{ } xs = ac * 10^{\text{length } xs} + \text{número } xs}$$

Más adelante...

- ▶ Implicaciones y propiedades más complejas (práctica)
- ▶ Inducción estructural en Lógica Proposicional (próximas teóricas)
- ▶ Inducción estructural en Cálculo Lambda (próximas teóricas)
- ▶ Inducción en grafos, otros lenguajes, etc. (otras materias)
- ▶ Otros métodos de demostración (esta y otras materias)

i i i i i i i j j ? ? ? ? ? ? ? ?

Lectura recomendada

Capítulo 6 del libro de Bird.

Richard Bird. *Thinking functionally with Haskell*

Cambridge University Press, 2015.